

Concept Document

Name: Saloni Kumari

Date: 07 May 2026

Target: Concept

1. SQL Injection:

An attacker inserts malicious SQL code into an input field to manipulate the backend database.

How it works:

- Application does not validate/sanitize user input.
- Attacker replaces expected input with SQL commands.
- The command becomes part of the database query and executes.

Real-world case – TalkTalk (2015):

- Attackers exploited SQLi on a web page.
- Retrieved customer names, addresses, phone numbers (157,000 users affected).
- Company fined £400,000, stock price dropped, trust lost.

Impact:

Data theft, authentication bypass, server compromise.

2. Cross-Site Scripting (XSS):

Attacker injects malicious JavaScript into a trusted website. The script executes in other users' browsers.

Stored XSS:

- Malicious script is permanently stored on the server (database, comments, profile).
- Every user who views the infected page executes the script automatically.

Real-world – MySpace (2005):

- Samy injected a worm into his profile.
- It added him as a friend and self-replicated.
- Within 20 hours → 1 million+ infected accounts.

Lessons: sanitize user content, block unsafe JS, use CSP.

Reflected XSS:

- Script is not stored – it is reflected from user input.
- Delivered via malicious links, phishing emails.

Real-world – eBay (2014):

- Researchers found reflected XSS in listings.
- Attackers could craft URLs that executed JS in victim's browser.
- Could steal session cookies, redirect to fake login pages.

Impact of XSS:

Session hijacking, credential theft, defacement, phishing.

3. Command Injection

Attacker executes arbitrary operating system commands on the server through a vulnerable application.

How it works:

- App passes user input directly to a system shell (e.g., ping, nslookup).
- Attacker adds extra commands using metacharacters (;, &&, |)

Real-world – Shellshock (2014):

- Flaw in Bash shell. Attackers sent crafted HTTP requests to execute system commands.
- Millions of servers vulnerable; attackers gained remote control.

Impact:

Remote code execution, data theft, malware installation, full server compromise.

4. Authentication Flaws:

Weaknesses in the process that verifies a user's identity.

Normal authentication flow:

1. User enters credentials.
2. Server verifies them.
3. Server creates a session or token.
4. User gains access.

Common flaws:

- Weak password policies
- Lack of multi-factor authentication (MFA)
- Session fixation / session hijacking
- Insecure cookie handling (e.g., missing HttpOnly, Secure flags)

Real-world example – Session Hijacking:

Attacker steals an authentication token (via XSS or network sniffing) and impersonates the victim without needing credentials.

Impact:

Unauthorized access, account takeover, privilege escalation.

5. File Inclusion:

Attacker tricks an application into loading an unintended file, often via a parameter like ?page=home.php.

Types:

- i. **Local File Inclusion (LFI)** – includes a file already on the server.
- ii. **Remote File Inclusion (RFI)** – loads a malicious file from an external server.

How it works:

Dynamic inclusion without proper validation – attacker manipulates the file path.

Real-world:

Many legacy PHP apps have suffered RFI/LFI, leading to remote code execution and data leaks.

Impact:

Sensitive file disclosure, remote code execution, server compromise.

Summary Table:

Vulnerability	Core Risk	Real Example	Fix
SQL Injection	Data theft, login bypass	TalkTalk(2015)	Parameterized queries
Stored XSS	Mass infection, data theft	MySpace(2005)	Sanitize + CSP

Reflected XSS	Session hijection	eBay(2014)	Output encoding
Command Injection	Server takeover	Shellshock(2014)	Avoid system calls
Aunthentication Flaws	Account takeover	Session hijacking	MFA + secure cookies
File Inclusion	File read,RCE	PHP LFI/RFI Attacks	Whitelist + input validation